

Building Generative AI-Powered Applications with Python

Maulana Hafidz Ismail

21 Oktober 2025

Contents

1	Image Captioning with Generative AI	1
1.1	Model-model generatif AI yang menjadi dasar dalam pengembangan aplikasi generatif AI	1
1.1.1	Variational Autoencoders (VAEs)	1
1.1.2	Generative Adversarial Networks (GANs)	1
1.1.3	Transformer-based Models	1
1.1.4	Diffusion Models	2
1.2	Definisi Foundation Model	2
1.3	Karakteristik Foundation Models	2
1.4	Kemampuan Foundation Models	2
1.5	Contoh Foundation Models	2
1.6	Adaptasi Foundation Models	3
1.7	Keterbatasan Foundation Models	3
1.8	Pengenalan Image Captioning AI	3
1.9	Hugging Face Platform	3
1.10	Hugging Face Transformers	4
1.11	Pengenalan Bootstrapping Language-Image Pretraining (BLIP)	4
1.12	Menggunakan BLIP pada Hugging Face	5
1.13	Pengenalan Gradio	6
1.13.1	Mulai Menggunakan Gradio	7
1.13.2	Klasifikasi Gambar dengan PyTorch	9
2	Create Your Own ChatGPT-Like Website	12
2.1	Pengantar Chatbot	12
2.2	Komponen Utama Chatbot	12
2.3	Pemilihan LLM	12
2.4	Pengembangan Chatbot	12
2.5	Tokenizer	12
3	Create a Voice Assistant	13
4	Generative AI-Powered Meeting Assistant	14
5	Summarize Your Private Data with Generative AI and RAG	15
6	Babel Fish (Universal Language Translator) with LLM and SST TTS	16
7	Build and AI Career Coach	17

1 Image Captioning with Generative AI

1.1 Model-model generatif AI yang menjadi dasar dalam pengembangan aplikasi generatif AI

1.1.1 Variational Autoencoders (VAEs)

VAEs adalah model generatif yang digunakan untuk mengurangi dimensi data dan menghasilkan versi baru dari data tersebut. Dan fiturnya sebagai berikut:

- Bekerja dengan berbagai jenis data seperti gambar, teks, dan audio.
- Menggunakan encoder untuk mempelajari distribusi probabilitas dari data input dan menyimpannya dalam latent space, yang merupakan representasi terkompresi dari data.

1.1.2 Generative Adversarial Networks (GANs)

GANs adalah model yang terdiri dari dua jaringan saraf konvolusional (CNN) yang bersaing satu sama lain untuk menghasilkan data yang realistis. Fiturnya sebagai berikut:

- Satu CNN berfungsi sebagai generator yang menghasilkan sampel data, sementara yang lain berfungsi sebagai discriminator yang membedakan antara sampel nyata dan palsu.
- Dapat digunakan untuk menghasilkan gambar baru, melakukan transfer gaya, dan bahkan membuat deep fakes.

1.1.3 Transformer-based Models

Model berbasis transformer adalah arsitektur yang dirancang untuk mengatasi masalah vanishing gradients pada RNN, dengan menggunakan mekanisme perhatian untuk memfokuskan pada bagian penting dari teks. Berikut fiturnya:

- Mampu memodelkan ketergantungan jangka panjang dalam teks.
- Digunakan dalam pemrosesan bahasa alami dan dapat menghasilkan teks yang koheren dan relevan secara kontekstual.

Recurrent Neural Network (RNN) adalah jenis jaringan saraf yang dirancang untuk memproses data sekuensial dengan cara yang mempertahankan informasi dari langkah sebelumnya. Ini berarti RNN dapat "mengingat" informasi dari input sebelumnya dan menggunakannya untuk mempengaruhi output saat ini. Informasi lain terkait RNN:

- Memori Jangka Pendek: RNN memiliki kemampuan untuk menyimpan informasi dari langkah sebelumnya, yang membuatnya cocok untuk tugas-tugas seperti analisis teks, pengenalan suara, dan prediksi urutan.

- Vanishing Gradient Problem: Namun, RNN sering mengalami masalah yang disebut vanishing gradients, di mana informasi dari langkah-langkah yang lebih awal dalam urutan menjadi semakin sulit untuk dipertahankan saat jaringan dilatih. Ini adalah salah satu alasan mengapa model berbasis transformer dikembangkan.

1.1.4 Diffusion Models

Model difusi adalah model generatif yang mengatasi penurunan informasi akibat noise dalam latent space dengan menerapkan prinsip difusi. Berikut fiturnya:

- Melibatkan dua langkah: forward diffusion (menambahkan noise) dan reverse diffusion (menghilangkan noise untuk memulihkan data).
- Contoh model ini termasuk OpenAI's Dall-E2 dan Stability AI's Stable Diffusion XL, yang menghasilkan konten grafis berkualitas tinggi.

1.2 Definisi Foundation Model

Foundation Model adalah model yang dilatih pada sejumlah besar data dan dapat disesuaikan untuk berbagai aplikasi. Ini adalah model umum yang memiliki kemampuan multimodal atau dapat menerima banyak bentuk input seperti gambar, teks, suara, dll.

1.3 Karakteristik Foundation Models

- Pre-training: Proses di mana model dilatih menggunakan algoritma tanpa pengawasan pada data yang tidak diberi label, memungkinkan model untuk menghubungkan informasi yang beragam secara mandiri.
- Multimodal: Kemampuan untuk menerima input dalam berbagai format seperti teks, gambar, audio, atau video.

1.4 Kemampuan Foundation Models

- Generative AI: Semua foundation models memiliki kemampuan generatif, tetapi tidak semua model generatif adalah foundation models.
- Large Language Models (LLMs): Foundation models yang dilatih pada basis data pemrosesan bahasa alami yang besar, seperti GPT-3 dan GPT-4.

1.5 Contoh Foundation Models

- OpenAI's Dall-E: Model yang dapat melakukan berbagai tugas terkait gambar.
- Google's Bert: Model yang dilatih pada parameter besar untuk pemrosesan bahasa.

1.6 Adaptasi Foundation Models

Foundation models dapat disesuaikan untuk banyak aplikasi, memungkinkan bisnis kecil untuk membuat model AI generatif yang lebih efisien dan terjangkau.

1.7 Keterbatasan Foundation Models

- **Bias:** Output yang dihasilkan dapat terpengaruh oleh bias dalam data pelatihan.
- **Hallucination:** LLMs dapat menghasilkan informasi yang salah karena salah menginterpretasikan konteks data.

1.8 Pengenalan Image Captioning AI

- Image Captioning AI: Teknologi yang mengubah informasi visual dari gambar menjadi bahasa yang dapat dibaca mesin. Ini membantu dalam meningkatkan aksesibilitas, hasil pencarian, dan keamanan.
- Manfaat:
 - Memungkinkan penemuan konten yang lebih dalam, meningkatkan kehadiran di media sosial, dan manajemen data yang efisien.
 - Bertujuan untuk membangun alat AI yang tidak hanya melihat gambar, tetapi juga memahaminya dan membuat file teks yang memberikan deskripsi bermakna tentang isi gambar.
 - Dengan memberikan deskripsi yang tepat, pencarian gambar menjadi lebih mudah dan mengurangi beban kerja.
- BLIP (Bootstrapping Language-Image Pre-training): Model yang dapat melakukan berbagai tugas multi-modal, termasuk pengambilan gambar-teks dan captioning gambar.

1.9 Hugging Face Platform

Hugging Face adalah platform open source untuk kecerdasan buatan (AI) yang memungkinkan kolaborasi antara ilmuwan, pengembang, dan bisnis untuk membangun alat machine learning yang dipersonalisasi.

Platform ini bertujuan untuk menciptakan pusat bagi komunitas AI open source untuk berbagi model, dataset, dan aplikasi, sehingga AI dapat diakses oleh semua jenis pengguna.

Hugging Face memiliki lebih dari 250,000 open models, 50,000 datasets, dan satu juta open demos yang terus berkembang.

Platform ini menyediakan library transformer open source dengan lebih dari 25,000 pretrained models untuk PyTorch, TensorFlow, dan Google Jax.

- PyTorch: Library deep learning yang digunakan untuk membangun model AI.

- TensorFlow: Platform machine learning yang digunakan untuk membangun dan melatih model AI.
- Google Jax: Framework machine learning yang memungkinkan komputasi yang lebih efisien.

1.10 Hugging Face Transformers

Hugging Face Transformers adalah perpustakaan sumber terbuka yang populer yang menyediakan model dan alat pemrosesan bahasa alami (NLP) terkini. Perpustakaan ini menawarkan berbagai model yang telah dilatih sebelumnya untuk berbagai tugas NLP, termasuk klasifikasi teks, jawaban pertanyaan, dan terjemahan bahasa.

Salah satu fitur utama Hugging Face Transformers adalah dukungannya untuk pembelajaran multimodal, yang menggabungkan data teks dan gambar untuk tugas-tugas seperti penandaan gambar dan jawaban pertanyaan visual. Kemampuan ini sangat relevan dalam pembahasan Bootstrapping Language-Image Pretraining (BLIP), karena memanfaatkan data teks dan gambar untuk meningkatkan pemahaman dan kemampuan generasi deskripsi gambar oleh model AI.

Dalam pembahasan ini, kita akan menjelajahi cara menggunakan Hugging Face Transformers, khususnya model BLIP, untuk penulisan keterangan gambar dalam Python. Kita akan menunjukkan cara memuat model yang telah dilatih sebelumnya, memproses gambar, dan menghasilkan keterangan, sekaligus memperlihatkan kemampuan perpustakaan ini dalam menjembatani kesenjangan antara bahasa alami dan konten visual.

1.11 Pengenalan Bootstrapping Language-Image Pretraining (BLIP)

BLIP mewakili kemajuan signifikan di persimpangan antara pemrosesan bahasa alami (NLP) dan penglihatan komputer. BLIP, yang dirancang untuk meningkatkan model kecerdasan buatan (AI), memperkuat kemampuannya dalam memahami dan menghasilkan deskripsi gambar. Sistem ini belajar mengasosiasikan gambar dengan teks yang relevan, sehingga dapat menghasilkan keterangan gambar, menjawab pertanyaan terkait gambar, dan mendukung pencarian berbasis gambar.

BLIP penting untuk beberapa alasan:

- Pemahaman yang lebih mendalam: Ini memberikan pemahaman yang lebih rinci tentang konten dalam gambar, melampaui pengenalan objek untuk memahami adegan, tindakan, dan interaksi.
- Pembelajaran multimodal: Dengan mengintegrasikan data teks dan gambar, BLIP memfasilitasi pembelajaran multimodal, yang lebih mirip dengan cara manusia memandang dunia.

- Aksesibilitas: Membuat deskripsi gambar yang akurat dapat membuat konten lebih mudah diakses oleh orang dengan gangguan penglihatan.
- Pembuatan konten: Ini mendukung upaya kreatif dan pemasaran dengan menghasilkan teks deskriptif untuk konten visual, menghemat waktu, dan meningkatkan kreativitas.

Aplikasi praktis BLIP adalah dalam pengembangan sistem penandaan foto otomatis. Sistem semacam ini dapat digunakan di berbagai bidang. Sistem ini meningkatkan platform media sosial dengan menyarankan keterangan foto secara otomatis untuk foto yang diunggah. Sistem ini juga membantu sistem manajemen aset digital dengan menyediakan deskripsi yang dapat dicari untuk gambar yang disimpan.

1.12 Menggunakan BLIP pada Hugging Face

Hugging Face menyediakan platform untuk bereksperimen dengan BLIP dan model AI lainnya. Berikut adalah contoh cara menggunakan BLIP untuk penandaan gambar (image captioning) dalam Python.

Pastikan Anda telah menginstal Python dan perpustakaan Transformers. Jika belum, Anda dapat menginstal perpustakaan Transformers menggunakan pip. Lihat kode berikut.

```
# Install the transformers library
!pip install transformers Pillow torch torchvision
torchaudio

from transformers import BlipProcessor,
    BlipForConditionalGeneration
from PIL import Image

# Initialize the processor and model from Hugging Face
processor = BlipProcessor.from_pretrained("Salesforce/blip-
    image-captioning-base")
model = BlipForConditionalGeneration.from_pretrained("
    Salesforce/blip-image-captioning-base")

# Load an image
image = Image.open("path_to_your_image.jpg")

# Prepare the image
inputs = processor(image, return_tensors="pt")

# Generate captions
outputs = model.generate(**inputs)
caption = processor.decode(outputs[0], skip_special_tokens=
    True)

print("Generated Caption:", caption)
```

BLIP juga dapat menjawab pertanyaan tentang isi sebuah gambar. Lihat kode berikut.

```
import requests
from PIL import Image
from transformers import BlipProcessor,
    BlipForConditionalGeneration

# Load BLIP processor and model
processor = BlipProcessor.from_pretrained("Salesforce/blip-
    image-captioning-large")
model = BlipForConditionalGeneration.from_pretrained("
    Salesforce/blip-image-captioning-large")

# Image URL
img_url = 'https://storage.googleapis.com/sfr-vision-
    language-research/BLIP/demo.jpg'
raw_image = Image.open(requests.get(img_url, stream=True).
    raw).convert('RGB')

# Specify the question you want to ask about the image
question = "What is in the image?"

# Use the processor to prepare inputs for VQA (image +
    question)
inputs = processor(raw_image, question, return_tensors="pt")

# Generate the answer from the model
out = model.generate(**inputs)

# Decode and print the answer to the question
answer = processor.decode(out[0], skip_special_tokens=True)
print(f"Answer: {answer}")
```

BLIP dari Hugging Face Transformers membuka peluang baru untuk aplikasi kecerdasan buatan (AI) dengan memungkinkan pemahaman yang lebih mendalam terhadap konten visual dan deskripsi teks. Dengan menggunakan BLIP, pengembang dan peneliti dapat menciptakan aplikasi yang lebih intuitif, mudah diakses, dan menarik yang menjembatani kesenjangan antara dunia visual dan bahasa alami.

1.13 Pengenalan Gradio

Gradio adalah cara untuk mendemonstrasikan Machine Learning Model Anda dengan antarmuka web yang ramah pengguna sehingga siapa pun dapat menggunakannya di mana saja. Gradio adalah open source Python package yang memungkinkan Anda dengan cepat membangun demo atau aplikasi web untuk model pembelajaran mesin, API, atau fungsi Python apa pun. Anda kemudian dapat membagikan tautan ke demo atau aplikasi web Anda menggunakan fitur

berbagi bawaan Gradio. Tidak diperlukan pengalaman dalam JavaScript, CSS, atau hosting web.

Alasan menggunakan Gradio:

- Kemudahan penggunaan: Gradio memungkinkan pembuatan antarmuka untuk model dengan hanya beberapa baris kode.
- Kekuatan adaptasi: Gradio mendukung berbagai jenis input dan output, seperti teks, gambar, file, dan lainnya.
- Berbagi dan kolaborasi: Antarmuka dapat dibagikan dengan orang lain melalui URL unik, memudahkan kolaborasi dan pengumpulan umpan balik.

1.13.1 Mulai Menggunakan Gradio

Untuk mulai menggunakan Gradio, Anda perlu menginstal perpustakaan terlebih dahulu. Anda dapat menginstal Gradio menggunakan pip:

```
pip install gradio
```

Kemudian inisialisasi Gradio:

```
import gradio as gr

def greet(name, intensity):
    return "Hello, " + name + "!" * int(intensity)

demo = gr.Interface(
    fn=greet,
    inputs=["text", "slider"],
    outputs=["text"],
)

demo.launch(server_name="127.0.0.1", server_port= 7860)
```

Perhatikan bahwa untuk membuat demo pertama Anda, Anda telah membuat instance dari kelas `gr.Interface`. Kelas Interface dirancang untuk membuat demo untuk model pembelajaran mesin yang menerima satu atau lebih input dan mengembalikan satu atau lebih output.

Kelas Interface memiliki tiga argumen inti:

- `fn`: Fungsi untuk membungkus antarmuka pengguna (UI) di sekitar
- `inputs`: Komponen Gradio yang akan digunakan untuk input. Jumlah komponen harus sesuai dengan jumlah argumen dalam fungsi Anda.
- `outputs`: Komponen Gradio yang akan digunakan untuk output. Jumlah komponen harus sesuai dengan jumlah nilai kembalian dari fungsi Anda.

Argumen `fn` bersifat fleksibel — Anda dapat melewatkan fungsi Python apa pun yang ingin Anda bungkus dengan UI. Dalam contoh di atas, Anda melihat fungsi yang relatif sederhana, tetapi fungsi tersebut dapat berupa apa saja, mulai dari generator musik hingga kalkulator pajak hingga fungsi prediksi dari model pembelajaran mesin yang telah dilatih sebelumnya.

Argumen input dan output menerima satu atau lebih komponen Gradio. Seperti yang akan kita lihat, Gradio menyertakan lebih dari 30 komponen bawaan (seperti komponen `gr.Textbox()`, `gr.Image()`, dan `gr.HTML()`) yang dirancang untuk aplikasi pembelajaran mesin.

Jika fungsi Anda menerima lebih dari satu argumen, seperti pada contoh di atas, berikan list komponen ke inputs, dengan setiap komponen input sesuai dengan salah satu argumen fungsi secara berurutan. Hal yang sama berlaku jika fungsi Anda mengembalikan lebih dari satu nilai: cukup berikan daftar komponen ke outputs. Fleksibilitas ini membuat kelas Interface menjadi cara yang sangat powerful untuk membuat demo. Mari kita buat antarmuka sederhana untuk model penandaan gambar. Model BLIP (Bootstrapped Language Image Pretraining) dapat menghasilkan teks keterangan untuk gambar. Berikut cara membuat antarmuka Gradio untuk model BLIP.

```
pip install transformers
pip install torch
```

```
import gradio as gr
from transformers import BlipProcessor,
    BlipForConditionalGeneration
from PIL import Image

processor = BlipProcessor.from_pretrained("Salesforce/blip-
    image-captioning-base")
model = BlipForConditionalGeneration.from_pretrained("
    Salesforce/blip-image-captioning-base")

def generate_caption(image):
    # Now directly using the PIL Image object
    inputs = processor(images=image, return_tensors="pt")
    outputs = model.generate(**inputs)
    caption = processor.decode(outputs[0],
        skip_special_tokens=True)
    return caption

def caption_image(image):
    """
    Takes a PIL Image input and returns a caption.
    """
    try:
        caption = generate_caption(image)
        return caption
    except Exception as e:
```

```

        return f"An error occurred: {str(e)}"

iface = gr.Interface(
    fn=caption_image,
    inputs=gr.Image(type="pil"),
    outputs="text",
    title="Image Captioning with BLIP",
    description="Upload an image to generate a caption."
)

iface.launch(server_name="127.0.0.1", server_port= 7860)

```

Di sini, kita menggunakan `BlipProcessor` dan `BlipForConditionalGeneration` dari pustaka `transformers` untuk membuat model picture captioning. Contoh ini menunjukkan cara membuat antarmuka web menggunakan Gradio, di mana parameter input menentukan gambar dan outputnya adalah teks caption yang dihasilkan. Parameter judul dan deskripsi memperkaya antarmuka dengan memberikan konteks dan petunjuk bagi pengguna.

Model penandaan gambar seperti BLIP merupakan alat yang sangat powerful di berbagai bidang, mulai dari membantu individu dengan gangguan penglihatan memahami konten gambar hingga mengorganisir dan mencari melalui perpustakaan foto besar dengan efisien. Membuat antarmuka Gradio untuk model semacam ini membuatnya dapat diakses oleh pengguna non-teknis untuk berinteraksi dan memanfaatkan teknologi ini. Misalnya, fotografer atau manajer aset digital dapat menggunakan aplikasi Anda untuk secara otomatis menghasilkan nama deskriptif untuk gambar mereka, meningkatkan kegunaan dan kemudahan pencarian dalam perpustakaan digital mereka.

1.13.2 Klasifikasi Gambar dengan PyTorch

Klasifikasi gambar merupakan tugas utama dalam pengolahan citra komputer. Pengembangan klasifikasi yang lebih baik untuk mengidentifikasi objek yang terdapat dalam sebuah gambar merupakan bidang penelitian yang aktif, karena memiliki aplikasi yang luas, mulai dari kendaraan otonom hingga pencitraan medis.

Model-model semacam ini sangat cocok digunakan bersama komponen input gambar Gradio. Dalam tutorial ini, kita akan membangun demo web untuk mengklasifikasikan gambar menggunakan Gradio. Seluruh aplikasi web dapat dibangun menggunakan Python.

Pertama, kita memerlukan model klasifikasi gambar. Untuk tutorial ini, kita akan menggunakan model Resnet-18 yang telah dilatih sebelumnya, karena model ini dapat diunduh dengan mudah dari PyTorch Hub. Anda dapat menggunakan model yang telah dilatih sebelumnya yang berbeda atau melatih model Anda sendiri.

```
import torch
```

```
model = torch.hub.load('pytorch/vision:v0.6.0', 'resnet18',
                        pretrained=True).eval()
```

Selanjutnya, kita perlu mendefinisikan fungsi yang menerima masukan pengguna, dalam hal ini berupa gambar, dan mengembalikan prediksi. Prediksi tersebut harus dikembalikan dalam bentuk dictionary, di mana key-nya adalah nama class dan value-nya adalah confidence probabilities. Nama-nama kelas akan dimuat dari berkas teks ini.

Dalam kasus model yang telah dilatih sebelumnya, bentuknya akan seperti ini:

```
import requests
from PIL import Image
from torchvision import transforms

# Download human-readable labels for ImageNet.
response = requests.get("https://git.io/JJkYN")
labels = response.text.split("\n")

def predict(inp):
    inp = transforms.ToTensor()(inp).unsqueeze(0)
    with torch.no_grad():
        prediction = torch.nn.functional.softmax(model(inp)[0],
                                                  dim=0)
        confidences = {labels[i]: float(prediction[i]) for i in
                       range(1000)}
    return confidences
```

Mari ulas, fungsinya mengambil satu parameter:

`inp`: the input image as a PIL image

Fungsi ini mengubah gambar input menjadi PIL Image dan kemudian menjadi tensor PyTorch. Setelah memproses tensor melalui model, fungsi ini mengembalikan prediksi dalam bentuk dictionary bernama `confidences`. Key-nya adalah label kelas, dan value-nya adalah confidence probabilities yang sesuai.

Pada bagian ini, kita mendefinisikan fungsi `predict` yang memproses gambar input untuk mengembalikan probabilitas prediksi. Fungsi ini pertama-tama mengubah gambar menjadi tensor PyTorch dan kemudian meneruskannya melalui model yang telah dilatih sebelumnya. Kami menggunakan fungsi `softmax` pada langkah akhir untuk menghitung probabilitas setiap kelas. Fungsi `softmax` sangat penting karena mengubah keluaran logit mentah dari model, yang dapat berupa angka real apa pun, menjadi probabilitas yang jumlahnya sama dengan 1. Hal ini memudahkan untuk menafsirkan keluaran model sebagai tingkat kepercayaan untuk setiap kelas.

Sekarang setelah fungsi prediktif kita sudah disiapkan, kita dapat membuat antarmuka Gradio untuknya.

Dalam hal ini, komponen input adalah komponen gambar seret dan lepas. Untuk membuat komponen ini, kita menggunakan `Image(type="pil")` yang membuat komponen dan menangani prapemrosesan untuk mengubahnya menjadi gambar PIL.

Komponen output akan berupa Label, yang menampilkan label teratas dalam bentuk yang rapi. Karena kita tidak ingin menampilkan semua 1.000 label kelas, kita akan menyesuaikannya untuk menampilkan hanya 3 kelas teratas dengan mengonfigurasinya sebagai `Label(num_top_classes=3)`.

Terakhir, kita akan menambahkan satu parameter lagi, parameter `examples`, yang memungkinkan kita untuk mengisi antarmuka dengan beberapa contoh yang telah ditentukan sebelumnya. Kode untuk Gradio terlihat seperti ini:

```
import gradio as gr
gr.Interface(fn=predict,
            inputs=gr.Image(type="pil"),
            outputs=gr.Label(num_top_classes=3),
            examples=["/content/lion.jpg", "/content/cheetah.jpg"]
            ).launch()
```

Contoh path sudah disediakan, `/content/lion.jpg` dan `/content/cheetah.jpg`, hanyalah tempat penampung. Anda harus mengganti paths ini dengan path aktual ke gambar-gambar di sistem atau server Anda tempat Anda menyimpan gambar-gambar yang ingin Anda gunakan untuk pengujian. Hal ini memastikan bahwa saat Anda atau orang lain menggunakan antarmuka Gradio, contoh-contoh tersebut dimuat dengan benar dan dapat digunakan untuk mendemonstrasikan fungsi klasifikasi gambar Anda.

Itu semua kode yang Anda butuhkan untuk membuat demo web untuk klasifikasi gambar. Jika Anda ingin berbagi dengan orang lain, coba atur `share=True` saat Anda menjalankan Interface.

Gradio memudahkan proses pembuatan demo web interaktif untuk model pembelajaran mesin. Dengan mengintegrasikan Gradio dengan model seperti BLIP untuk penandaan gambar, Anda dapat membuat aplikasi praktis dan ramah pengguna yang memanfaatkan kekuatan AI untuk menyelesaikan masalah dunia nyata. Alat ini tidak hanya membantu dalam menunjukkan kemampuan model Anda tetapi juga dalam mengumpulkan umpan balik berharga untuk perbaikan lebih lanjut.

2 Create Your Own ChatGPT-Like Website

2.1 Pengantar Chatbot

Chatbot adalah program komputer yang mensimulasikan percakapan manusia, memungkinkan pengguna untuk mendapatkan informasi dengan cepat.

Dengan teknologi AI, chatbot dapat memahami pertanyaan dan memberikan jawaban berdasarkan data yang telah dikumpulkan.

2.2 Komponen Utama Chatbot

- Transformer: Program yang berfungsi sebagai otak chatbot, mengelola pemrosesan data input dan output.
- LLM (Large Language Model): Model yang membantu chatbot memahami pertanyaan dan menghasilkan respons yang mirip dengan manusia.

2.3 Pemilihan LLM

Penting untuk memilih LLM yang sesuai dengan tujuan chatbot, seperti:

- GPT-2/GPT-3: Untuk generasi teks umum.
- BERT: Untuk analisis sentimen.
- T-5: Untuk terjemahan bahasa.

Pertimbangan lain dalam memilih LLM meliputi lisensi, ukuran model, data pelatihan, serta kinerja dan akurasi.

2.4 Pengembangan Chatbot

Anda akan menggunakan model BlenderBot dari Facebook, yang tersedia dengan lisensi sumber terbuka melalui Hugging Face.

Hugging Face's Python library transformers memudahkan interaksi dengan LLM dan mengoptimalkan input ke dalam format yang dapat dipahami oleh LLM, yaitu token.

2.5 Tokenizer

Tokenizer adalah komponen dalam Natural Language Processing (NLP) yang berfungsi untuk memecah teks menjadi unit-unit kecil yang disebut token. Token ini bisa berupa kata, potongan kata, atau karakter, tergantung jenis tokennya.

Token ini kemudian diubah menjadi representasi numerik agar model bisa memahaminya dan melakukan tugas seperti menjawab pertanyaan atau membuat teks.

3 Create a Voice Assistant

4 Generative AI-Powered Meeting Assistant

5 Summarize Your Private Data with Generative AI and RAG

6 Babel Fish (Universal Language Translator) with LLM and SST TTS

7 Build and AI Career Coach